

PortfolioCheck — Automated Verification of Formal Coursework Submission Requirements

Siva Kondaru · Matr. No. 10243569 · Project: Software Engineering (DLMCSPSE01) · Tutor: Holger Klus

Solution and Concept

Every portfolio submission is governed by formal provisions that are entirely mechanical: a naming pattern for the main folder, four subdirectories with exact names, permitted file formats, a size limit of 15 MB, a links file, and a code archive that must be identical to the public GitHub repository. These provisions carry marks, yet students verify them by hand in the last hours before a deadline, and errors surface only when they can no longer be corrected. Research on automated assessment concentrates on the correctness of program code; the formal envelope of a submission has received almost no attention.

PortfolioCheck closes this gap. A student uploads the zip folder they intend to submit and selects the portfolio phase. The web application evaluates the archive against seven rules derived from the assignment brief and returns a report in which every rule is marked *passed*, *to review* or *must fix*. Two principles define the concept. First, every negative verdict states the exact correction required — a report that only says “wrong” would reproduce the student’s problem. Second, every finding names the provision of the brief it derives from, so the student can check the verdict against the source. The tool judges form, never content, and stores nothing: an upload is deleted before the response is returned. The primary target group is students in distance-learning programmes, above all those submitting for the first time and those working close to a deadline; a student starting from nothing can also download an empty, correctly named folder structure.

Making of: Technical Approach

The project followed an incremental Kanban approach; Scrum was rejected because its events presuppose a team. After the Phase 2 feedback the board moved into GitHub Projects. Eleven work packages across the three portfolio phases were each linked to the requirements they deliver: thirteen functional and nine non-functional requirements, the latter structured along ISO/IEC 25010.

The architecture is layered around a domain model that depends on nothing. Each formal provision is one rule class registered with a rule registry, so a new rule changes no other component — a property demonstrated when the final rule was added. The implementation uses Python 3.11 and FastAPI with server-rendered templates; Django and a single-page front end were set aside because their database and second build pipeline would remain unused. Uploads are treated as hostile: archives with path traversal, absolute paths, excessive entry counts or extreme compression ratios are rejected before anything is extracted. The application runs as a Docker container under an unprivileged user and is deployed on Render.

The most distinctive rule verifies that the public repository and the submitted code archive are identical — a provision practically impossible to check by hand. It reads the repository address

from the links file, retrieves the file tree through the GitHub REST API, and compares both sides using git blob hashes, the content fingerprints git itself uses, so missing, surplus and modified files are detected without downloading any file. If GitHub cannot be reached, this rule degrades to a warning and all other rules still run. Code, tests, sample data and documentation are versioned together in one public repository, and the submitted state is marked by the release tag v1.0.0.

Testing and Results

The application comprises about 1,100 lines of code and 900 lines of tests. All 73 automated tests pass; they cover every rule against a compliant and a violating submission, the extraction defences, the GitHub client against a simulated service, and the complete path through the web interface. No test uses the network. Thirteen manual test cases specify concrete input files and the exact expected output; nine generated sample submissions serve as their input and as test data for anyone trying the application. All thirteen functional requirements are implemented, and every non-functional requirement has recorded evidence: a typical submission is checked well within the three-second target, a deliberately failing rule leaves the rest of the report intact, and one container image serves both local use and the cloud deployment. Every component of the UML diagram maps to a named module. Seven items of technical debt remain documented, among them a global rule registry and hard-coded limits.

Lessons Learned

Three lessons stand out. First, *defences before features*: implementing the archive safety checks before any rule meant no untrusted input ever passed an unhardened path, and nothing had to be retrofitted. Second, *isolate the external dependency early*: placing GitHub behind an interface let the rest of the system proceed independently and made the most complex rule fully testable offline. Third, *the feedback loop is part of the method*: the tutor's feedback corrected the architecture notation to UML, clarified that technical debt means implementation shortcuts rather than missing features, and showed that a Kanban board is evidence only when it lives in a tool linked to the commits, not when it is drawn afterwards. Writing each test together with its rule proved the most effective practice: when the final rule was added late, the existing suite confirmed at once that nothing else had broken.

Regarding time and resources, the repository comparison consumed the largest share of the final phase, as the risk register had anticipated; scheduling it first left enough buffer for the deployment.

Outlook

The most valuable next steps follow from the recorded technical debt. An explicit rule registry and externalised configuration would allow the rule set to be adapted to the briefs of other courses and institutions, and a cache for repository inventories would let one shared deployment serve a whole cohort in the days before a deadline.